

软件设计模式汇报稿

This speech script is tailored to your PPT slides, providing English narration with corresponding Chinese translations to help you prepare.

Slide 1: Title Page

English:

"Good morning/afternoon, everyone. We are Group 18. Today, we are excited to present our Refactoring Report on the 'Stardew Valley' project for our Software Design Patterns course. Our team members include Zhang Zhuhe, Qian Baoqiang, Tan Pengchong, and Li Siyuan."

Chinese (中文):

"大家早上好/下午好。我们是第18组。今天很高兴向大家展示我们软件设计模式课程的‘星露谷物语’项目重构报告。我们的团队成员包括张竹和、钱宝强、谭鹏翀和李思远。"

Slide 2: Project Overview

English:

"First, let's look at the project overview. Our project is a farming simulation game inspired by Stardew Valley. It features core gameplay like planting, ranching, exploration, and NPC interaction.

However, before refactoring, we faced significant challenges. The code had **high coupling** between the UI and game logic, severe **code duplication**, and several **performance bottlenecks**. To address these, we applied six design patterns: Factory Method, Flyweight, Observer, Command, Facade, and Object Pool."

Chinese (中文):

"首先来看项目概况。我们的项目是一款受星露谷物语启发的农场模拟游戏。它包含种植、养殖、探索和NPC交互等核心玩法。

然而，在重构之前，我们面临着巨大的挑战。代码中**UI与游戏逻辑高度耦合**，存在严重的**代码重复**，以及若干**性能瓶颈**。为了解决这些问题，我们应用了六种设计模式：工厂方法、享元、观察者、命令、外观和对象池。"

Slide 3: Factory Method

English:

"The first pattern we applied is the Factory Method.

The Problem: Our `StoreUI` class violated the Open/Closed Principle (OCP) because it relied on long `if-else` chains to create different animal types. Additionally, scene boundary coordinates were hardcoded and duplicated across multiple UI classes.

The Solution: We implemented an `AnimalFactory` that uses a registry map to create animal instances via string keys. We also created a `SceneBoundaryFactory`, a static factory providing centralized configuration data.

The Benefit: This successfully **decoupled** the UI from concrete animal classes and significantly improved **maintainability** by managing scene boundaries in one central place."

Chinese (中文):

"我们应用的第一个模式是工厂方法。

问题: 我们的 `StoreUI` 类违反了开闭原则 (OCP)，因为它依赖冗长的 `if-else` 链来创建不同的动物类型。此外，场景边界坐标被硬编码并重复出现在多个UI类中。

解决方案: 我们实现了一个 `AnimalFactory`，它使用注册表映射通过字符串键来创建动物实例。我们还创建了一个 `SceneBoundaryFactory`，这是一个提供集中配置数据的静态工厂。

收益: 这成功地将UI与具体的动物类**解耦**，并通过在一个中心位置管理场景边界，显著提高了**可维护性**。"

Slide 4: Flyweight Pattern

English:

"Next is the Flyweight Pattern.

The Problem: We had hundreds of `Item` objects declared globally, leading to memory waste. Each item instance redundantly stored duplicate data like its name, icon path, and price.

The Solution: We extracted the shared data into an `ItemMetadata` class representing the intrinsic state. We then used an `ItemFlyweightFactory` to manage these shared instances.

The Benefit: This optimization drastically reduced **memory usage** by storing metadata once per item type, not per instance. It also ensured **safe initialization**, solving the 'Static Initialization Order Fiasco'."

Chinese (中文):

"接下来是享元模式。

问题: 我们在全局声明了数百个 `Item` 对象，导致内存浪费。每个物品实例都冗余地存储了重复数据，如名称、图标路径和价格。

解决方案: 我们将共享数据提取到一个代表内在状态的 `ItemMetadata` 类中。然后，我们使

用 `ItemFlyweightFactory` 来管理这些共享实例。

收益： 这一优化通过每种物品类型只存储一份元数据而不是每个实例一份，大幅减少了内存使用。它还确保了安全初始化，解决了‘静态初始化顺序灾难’。”

Slide 5: Observer Pattern

English:

"For data synchronization, we used the Observer Pattern.

The Problem: Our scene classes were inefficiently polling `is_updated` flags every frame. This led to inconsistency, where UI components like the status bar failed to sync data changes automatically.

The Solution: We made our core systems, like `Inventory` and `EconomicSystem`, inherit from a `GameStateSubject`. UI components like `MiniBag` and `StoreUI` then inherited from `GameStateObserver`.

The Benefit: This removed CPU-intensive polling loops, improving **performance**. It also **decoupled** the data logic from the view logic, ensuring consistent updates."

Chinese (中文):

“为了数据同步，我们使用了观察者模式。

问题： 我们的场景类效率低下地每帧轮询 `is_updated` 标志位。这导致了不一致性，状态栏等UI组件无法自动同步数据变化。

解决方案： 我们让核心系统（如 `Inventory` 和 `EconomicSystem`）继承自 `GameStateSubject`。`MiniBag` 和 `StoreUI` 等UI组件则继承自 `GameStateObserver`。

收益： 这移除了占用CPU的轮询循环，提升了性能。它还解耦了数据逻辑与视图逻辑，确保了更新的一致性。”

Slide 6: Command Pattern

English:

"To handle input, we implemented the Command Pattern.

The Problem: Input logic was tightly coupled with game logic inside `Player` and `Scene` classes. Hardcoded key bindings made remapping keys impossible, causing rigidity.

The Solution: We introduced an `InputManager` as the invoker. We encapsulated actions into concrete command classes like `PlantCommand` and `MoveCommand`.

The Benefit: This achieved a clear **separation of concerns** between input handling and business logic. It also improved **extensibility**, making it easy to add new commands or support custom key bindings."

Chinese (中文):

“为了处理输入，我们实施了命令模式。

问题： 输入逻辑与 `Player` 和 `Scene` 类中的游戏逻辑紧密耦合。硬编码的按键绑定使得改键变得不可能，导致了僵化。

解决方案： 我们引入了 `InputManager` 作为调用者。我们将动作封装成具体的命令类，如 `PlantCommand` 和 `MoveCommand`。

收益： 这实现了输入处理与业务逻辑之间清晰的**关注点分离**。它还提高了**可扩展性**，使得添加新命令或支持自定义键位变得容易。”

Slide 7: Facade Pattern

English:

"We also applied the Facade Pattern to simplify interactions.

The Problem: Complex actions, like eating food, required the UI to access multiple subsystems simultaneously—Inventory, Energy, and Economy. This resulted in spaghetti code and violated the Law of Demeter.

The Solution: We created a `GameInteractionFacade` that provides a unified API with simple methods like `consumeFood()` and `sellItem()`.

The Benefit: This **simplified the client** side, as the UI only interacts with the Facade. It also ensured **consistency** by centralizing validation and state synchronization across different systems."

Chinese (中文):

“我们还应用了外观模式来简化交互。

问题： 复杂的动作（如吃食物）需要UI同时访问多个子系统——背包、体力和经济。这导致了面条代码，并违反了迪米特法则。

解决方案： 我们创建了一个 `GameInteractionFacade`，它提供了一个统一的API，包含 `consumeFood()` 和 `sellItem()` 等简单方法。

收益： 这**简化了客户端**，因为UI只与外观类交互。它还通过跨不同系统集中验证和状态同步，确保了一致性。”

Slide 8: Object Pool Pattern

English:

"Finally, we used the Object Pool Pattern for optimization.

The Problem: We faced performance lags due to the frequent creation and destruction of `Sprite` objects during crop growth. This also caused memory fragmentation.

The Solution: We implemented a `CropSpritePool` manager. It maintains a collection of initialized sprites and uses a reuse mechanism with `getOrCreateSprite()` to borrow

and `recycleSprite()` to return objects.

The Benefit: This eliminated the instantiation overhead during scene updates, resulting in much better **FPS stability**."

Chinese (中文):

"最后，我们使用了对象池模式进行优化。

问题： 由于作物生长过程中频繁创建和销毁 `Sprite` 对象，我们面临性能卡顿。这也导致了内存碎片化。

解决方案： 我们实现了一个 `CropSpritePool` 管理器。它维护一个初始化精灵的集合，并使用复用机制，通过 `getOrCreateSprite()` 借用对象，通过 `recycleSprite()` 归还对象。

收益： 这消除了场景更新时的实例化开销，从而带来了更好的**帧率稳定性**。"

Slide 9: Summary & AI Tools

English:

"Regarding our AI tool usage:

We utilized **GitHub Copilot** for code review and **Claude 4.0** for architecture analysis, identifying refactoring opportunities, and providing strategies. **Gemini 3.0** helped with knowledge comprehension and problem discussion.

In Conclusion:

Our refactoring efforts significantly improved **code quality** by reducing coupling and improving cohesion. We optimized **performance** through Flyweight and Object Pool patterns, and enhanced **maintainability** by replacing hardcoded logic with flexible design patterns."

Chinese (中文):

"关于我们的AI工具使用：

我们利用 **GitHub Copilot** 进行代码审查，利用 **Claude 4.0** 进行架构分析、识别重构机会并提供策略。**Gemini 3.0** 帮助我们进行知识理解和问题讨论。

总结：

我们的重构工作通过降低耦合度和提高内聚性显著提升了**代码质量**。我们通过享元和对象池模式优化了**性能**，并通过用灵活的设计模式替代硬编码逻辑增强了**可维护性**。"

Slide 10: Thanks

English:

"That concludes our presentation. Thank you for listening. We are now open to any questions."

Chinese (中文):

“我们的展示到此结束。感谢大家的聆听。现在欢迎提问。”